

2023.12.09



인공지능 추천시스템

국민대학교 소프트웨어융합대학원
K2022531 최혜정

Contents

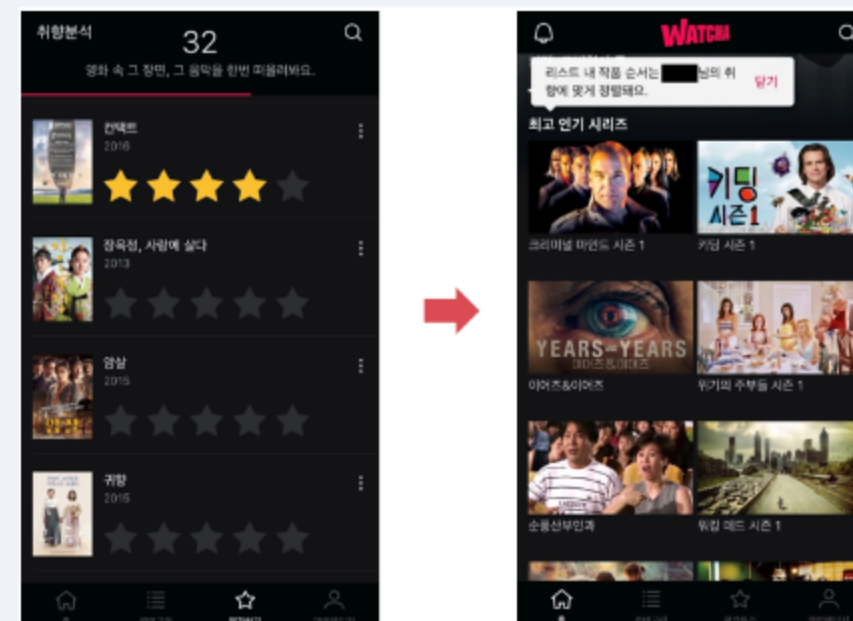
01	프로젝트 주제	프로젝트에서 수행하려는 과제와 사용한 데이터를 소개합니다.
02	추천 알고리즘 종류	다양한 추천 알고리즘 종류를 간단히 설명하고, 적합한 알고리즘을 선택합니다.
03	평가방법	추천 시스템의 여러 평가 방법을 간단히 설명하고, 프로젝트에 사용한 추천 시스템의 성능 평가에 대한 기준을 제시합니다.
04	설계 및 구현	사용한 기술 스택을 간략하게 소개하여 시스템의 설계와 구현에 대한 기본적인 내용을 설명합니다.
05	결과	프로젝트 결과물을 시연하고, 개선방향을 제시합니다.

1. 프로젝트 주제

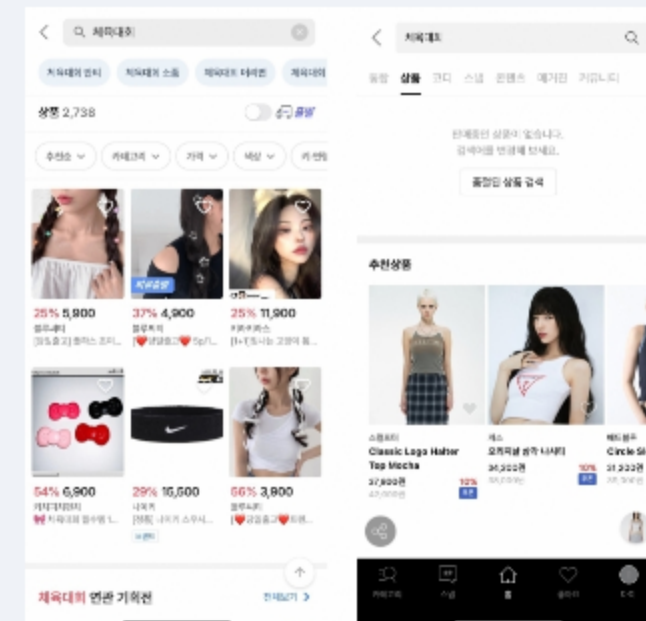
추천시스템 이란?



음악추천



영화추천

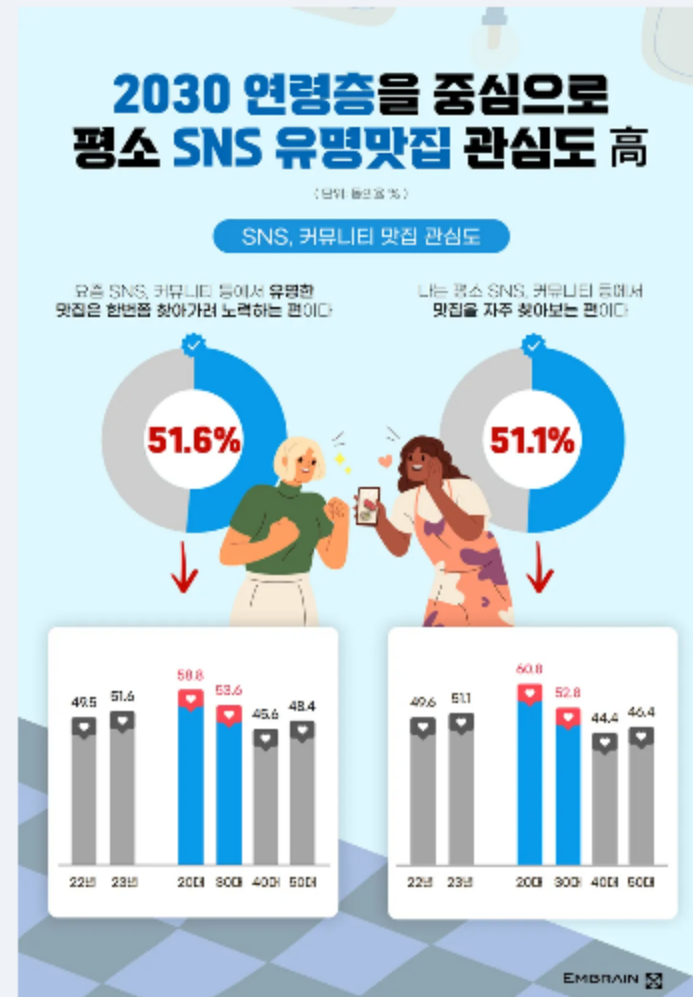


상품추천

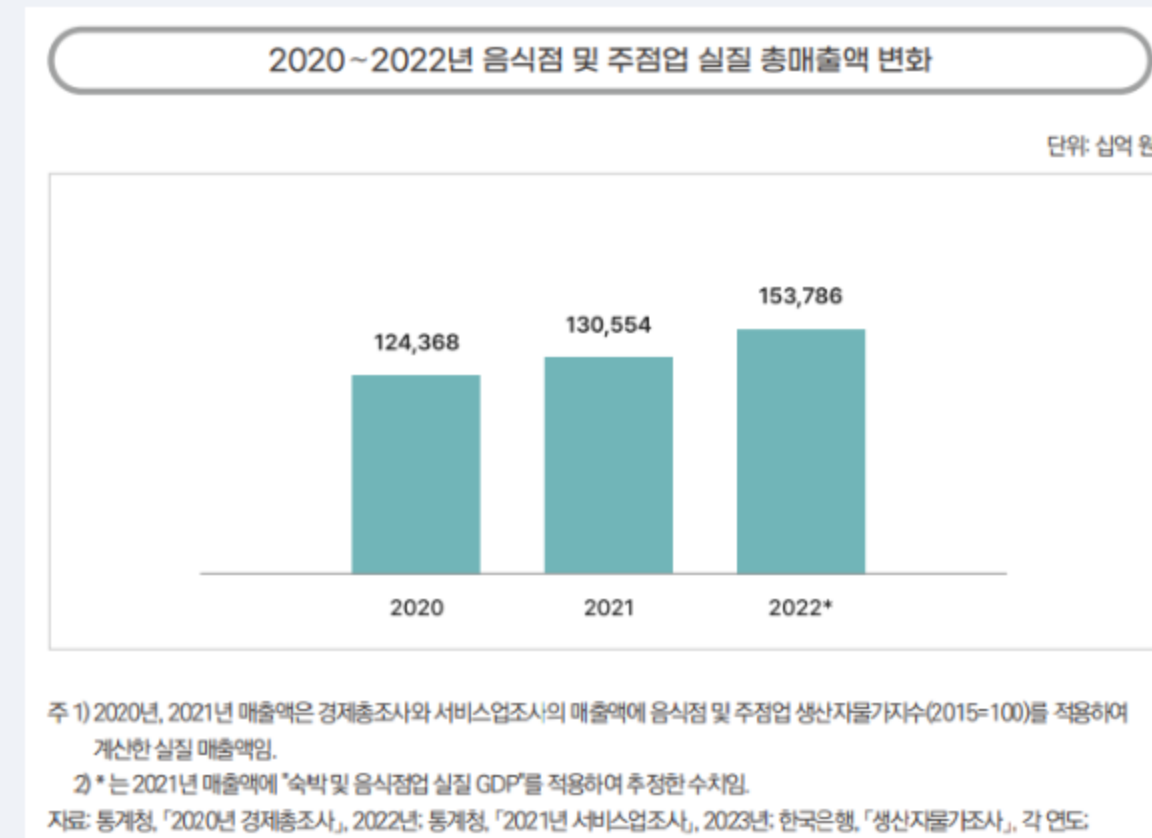
* 추천 시스템은 사용자가 관심을 가질만한 정보를 제공하는 시스템으로, 긍/부정을 수치로 나타낸 명시적 피드백 또는 직관적으로 알 수 없지만 사용자 기록을 바탕으로 암시적 피드백을 사용합니다.

1. 프로젝트 주제

주제 : 레스토랑 추천 시스템



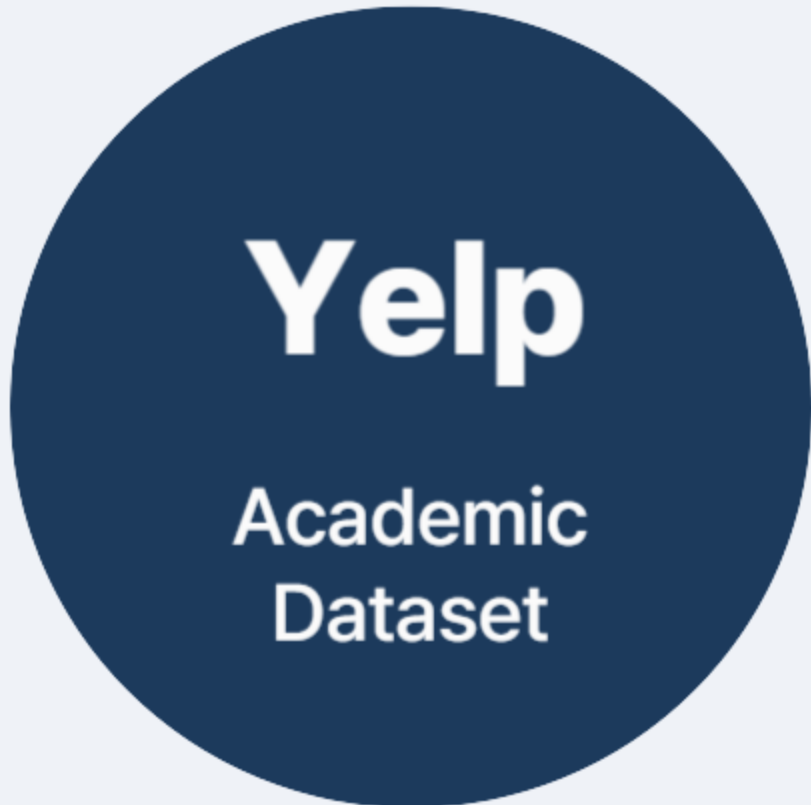
[TK_202301_TRY8724] 2023 파인다이닝, 오마카세 등 방문 니즈 관련 조사 (출처 : 트렌드 모니터)



2023년 제 8호, 2022년 외식업 주요 동향 및 특징 (출처 : 한국농촌경제연구원)

1. 프로젝트 주제

사용 데이터

The logo consists of a dark blue circle containing the word "Yelp" in white, bold, sans-serif font. Below "Yelp", the words "Academic" and "Dataset" are stacked in a smaller, white, sans-serif font.

Yelp

Academic
Dataset

- 1) **Business.json** - 레스토랑의 위치, 카테고리 및 등급 등 레스토랑 데이터
- 2) **Review.json** - 사용자가 작성한 특정 기업 평가에 대한 리뷰 데이터
- 3) **User.json** - 사용자 이름, 리뷰 수 등의 사용자 정보 데이터
- 4) **Checkin.json** - 업무 시간 및 해당 시간에 들어오는 인원 수 데이터
- 5) **Tip.json** - 짧은 리뷰 데이터
- 6) **Photos.json** - 사진, 업체 ID, 사진의 캡션 및 해당 카테고리 데이터

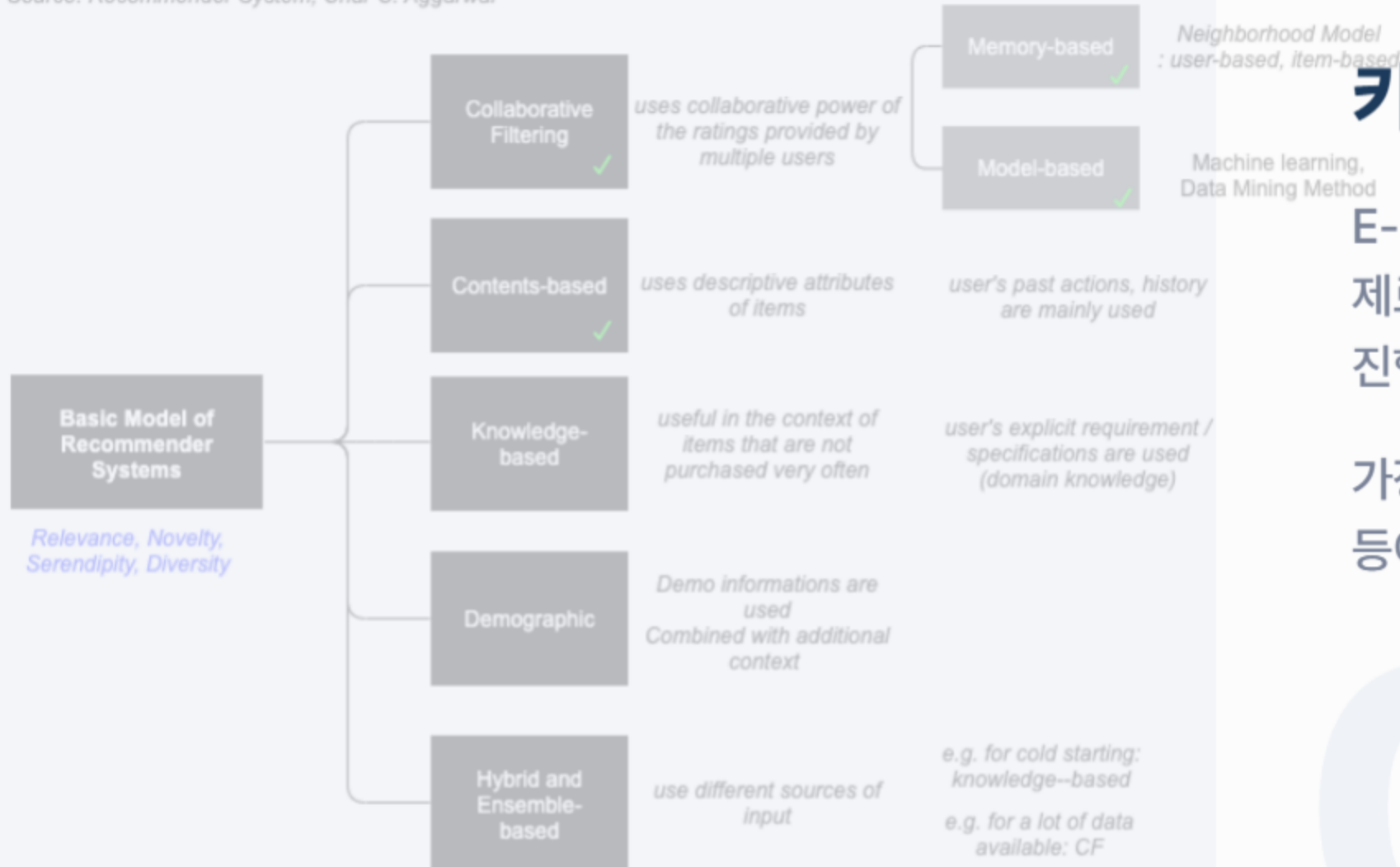
2. 추천 알고리즘 종류

키워드 : 개인화 추천, 리뷰 기반 추천, 추천시스템

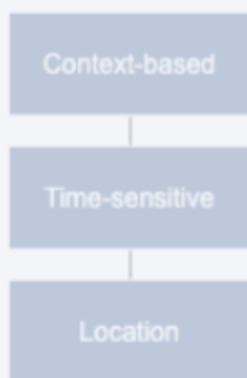
E-Commerce의 급격한 성장으로 정보의 과부하 현상이 심해짐에 따라, 소비자가 실제로 관심을 가질 가능성이 큰 상품을 선별하여 제안하는 추천시스템의 연구가 활발히 진행되고 있습니다.

가장 간단한 추천 방식으로는 베스트셀러기반 추천방법, 최소질의대상 상품결정 방법 등이 있습니다.

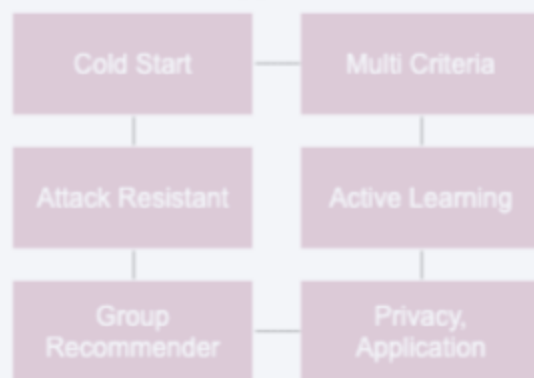
Source: Recommender System, Char C. Aggarwal



Domain Specific Challenge



Advanced Topic & Application

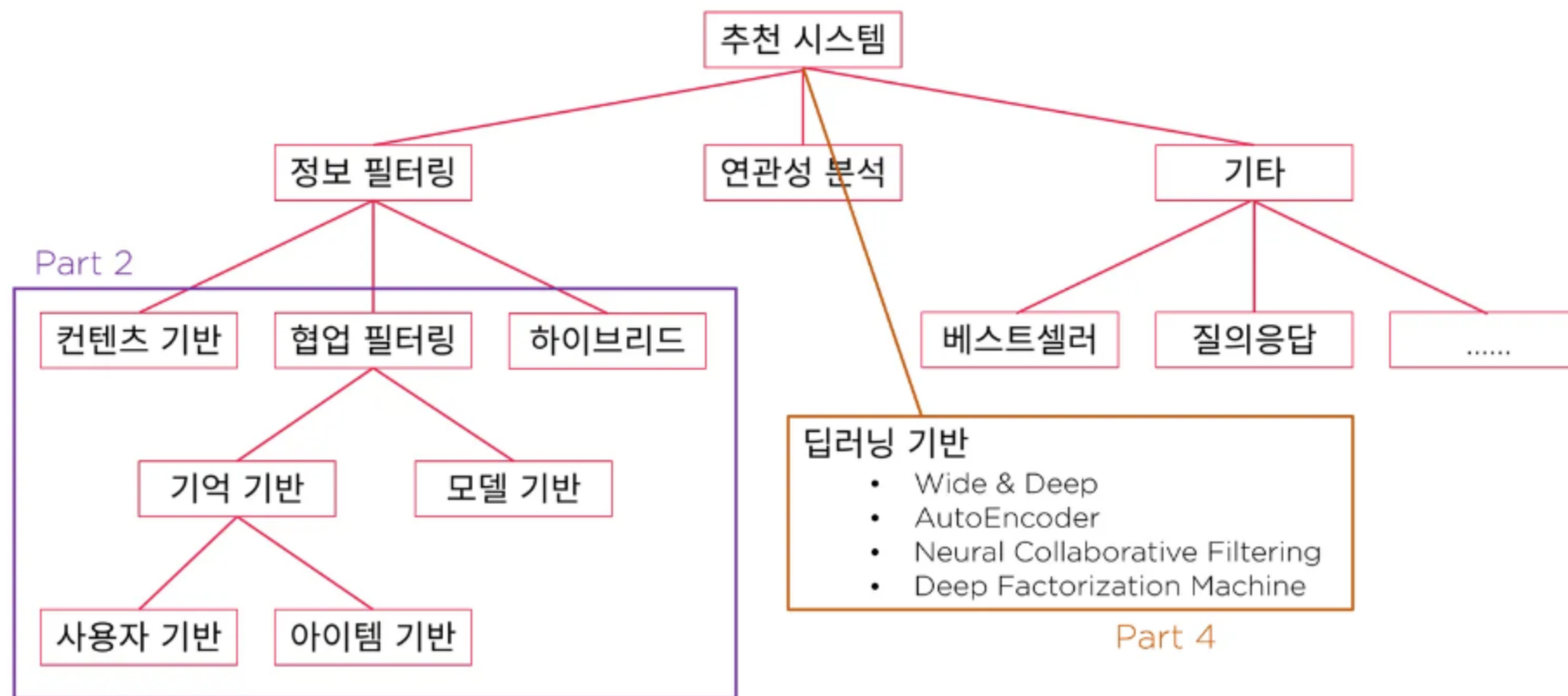


01 베스트 셀러 기반 추천

- 협력필터링 방식을 보완
- 고객의 정보 없이도 단시간 내에 상품을 추천 가능
- 고객에게 맞춤화되고 개인화된 추천을 할 수 없음
- 추천 정확도가 낮음

교보문고 10월 4주 베스트셀러			
순위	도서명	저자	출판사명
1	트렌드 코리아 2023	김난도	미래의창
2	사랑은 그렇게 하는 것이 아니다	김달	빅파시
3	하얼빈	김훈	문학동네
4	아버지의 해방일지	정지아	창비
5	불편한 편의점 (40만부 기념 빛깔 에디션)	김호연	나무열의자
6	불편한 편의점 2 (단풍 에디션)	김호연	나무열의자
7	역행자	자청	웅진지식하우스
8	반짝이는 하루, 그게 오늘이야	레슬리 마샬	미디어숲
9	마흔에 읽는 나체	장재형	유노북스
10	잘될 수밖에 없는 너에게	최서영	북로망스
11	단순한 열정	아니 에르노	문학동네
12	세상의 마지막 가차역	무라세 다케시	모모
13	원형(The One Thing) (리커버 특별판)	게리 켈리	비즈니스북스
14	세상에서 가장 쉬운 본질육아	지나영	21세기북스
15	파친코 1	이민진	애플루엔셀
16	부역 치트키	김성공	토네이도
17	원피스 103: 해방의 전사	Eiichiro Oda	대원씨아이
18	이 책은 돈 버는 법에 관한 이야기	고명환	라금
19	심리학이 조조에게 말한다 1	천위안	리드리드출판
20	부자 아빠 가난한 아빠 1 (20주년 특별 기념판)	로버트 기요사키	민음인

2. 추천 알고리즘 종류



02

컨텐츠 기반 추천

- 유저가 좋아했던 아이템 중 일부/전체와 비슷한 아이템 선정 (아이템 간의 점수, 유저-아이템 간의 프로파일링)

03

협업 필터링

- 비슷한 성향/취향을 가진 다른 유저가 좋아한 아이템을 현재 유저에게 추천

04

하이브리드 추천

- Content-based와 Collaborative Filtering의 장단점 보완 (cold-start 문제에 도움 줄 수 있음)

2. 추천 알고리즘 종류

02 컨텐츠 기반 추천

- 핵심은 콘텐츠(Items)들을 벡터 형태로 표현해야 하는 것.
- 가지고 있는 콘텐츠(item)들을 벡터로 표현하고, 벡터간의 유사도를 계산.
- 가장 비슷한 벡터들을 콘텐츠(item)으로 다시 바꿔서 추천.

유사도 함수

- 유사도 함수는 자신과 가장 비슷한 콘텐츠를 찾아 줄 때 사용
- 대표적으로 많이 사용하는 유사도 함수인 유클리디안, 코사인, 피어슨, 자카드 유사도 등이 있음.



벡터

Items를 벡터 형태로 표현. 도메인에 따라 다른 방법이 적용

item1
.
.
.
itemN

벡터1
.
.
.
벡터N

벡터들간의 유사도를 계산

벡터1부터 N까지
자신과 유사한
벡터를 추출

☑ 유클리디안 유사도

- 문서간의 유사도를 계산

	과일이	길고	노란	먹고	바나나	사과	싫은	저는	좋아요
문서1	0	0	0	1	0	1	1	0	0
문서2	0	0	0	1	1	0	1	0	0
문서3	0	1	1	0	2	0	0	0	0
문서4	1	0	0	0	0	0	0	1	1

```
1 from sklearn.metrics.pairwise import euclidean_distances
2 1 / (euclidean_distances(countvect_df, countvect_df) + 1e-5)
array([[1.00000000e+05, 7.07101781e-01, 3.33332222e-01, 4.08246624e-01],
       [7.07101781e-01, 1.00000000e+05, 4.47211596e-01, 4.08246624e-01],
       [3.33332222e-01, 4.47211596e-01, 1.00000000e+05, 3.33332222e-01],
       [4.08246624e-01, 4.08246624e-01, 3.33332222e-01, 1.00000000e+05]])
```

유클리디안 유사도 = $1 / (\text{유클리디안 거리} + 1e-05)$

$$\|p - q\| = \sqrt{(p - q) \cdot (p - q)} = \sqrt{\|p\|^2 + \|q\|^2 - 2p \cdot q}$$

$$\frac{1}{\sqrt{1^2 + 1^2 + 1e-5}} = 0.707$$

[장점]

- 계산하기가 쉬움

[단점]

- p와 q의 분포가 다르거나 범위가 다른 경우에 상관성을 놓침

☑ 코사인 유사도

- 문서간의 유사도를 계산

	과일이	길고	노란	먹고	바나나	사과	싫은	저는	좋아요
문서1	0	0	0	1	0	1	1	0	0
문서2	0	0	0	1	1	0	1	0	0
문서3	0	1	1	0	2	0	0	0	0
문서4	1	0	0	0	0	0	0	1	1

```
1 from sklearn.metrics.pairwise import cosine_similarity
2 cosine_similarity(countvect_df, countvect_df)
array([[1., 0.66666667, 0., 0.],
       [0.66666667, 1., 0.47140452, 0.],
       [0., 0.47140452, 1., 0.],
       [0., 0., 0., 1.]])
```

$$\frac{1 + 1}{\sqrt{1^2 + 1^2 + 1^2} \cdot \sqrt{1^2 + 1^2 + 1^2}} = 0.6667$$

[장점]

- 벡터의 크기가 중요하지 않은 경우에 거리를 측정하기 위한 메트릭으로 사용. (예 : 문서내에서 단어의 빈도수 - 문서들의 길이가 고르지 않더라도 문서내에서 얼마나 나왔는지라는 비율을 확인하기 때문에 상관없음.)

[단점]

- 벡터의 크기가 중요한 경우에 대해서 잘 작동하지 않음

2. 추천 알고리즘 종류

02 컨텐츠 기반 추천

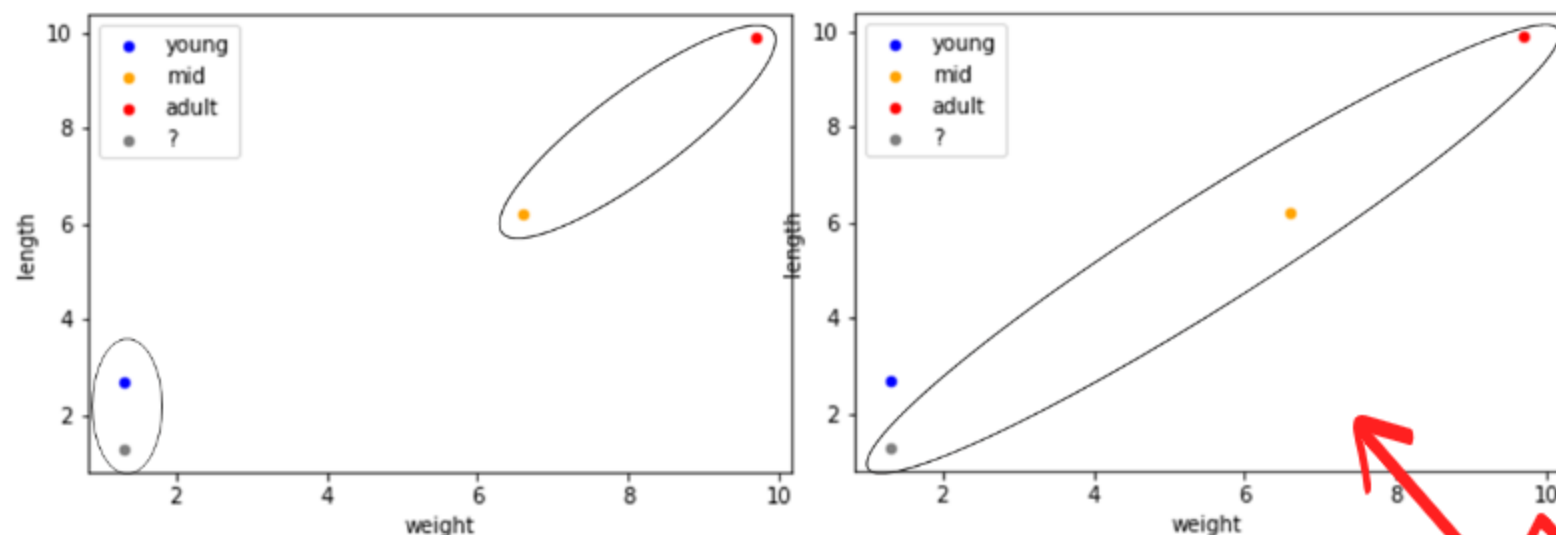
- 핵심은 컨텐츠(Items)들을 벡터 형태로 표현해야 하는 것.
- 가지고 있는 컨텐츠(item)들을 벡터로 표현하고, 벡터간의 유사도를 계산.
- 가장 비슷한 벡터들을 컨텐츠(item)으로 다시 바꿔서 추천.

유사도 함수

- 유사도 함수는 자신과 가장 비슷한 컨텐츠를 찾아 줄 때 사용
- 대표적으로 많이 사용하는 유사도 함수인 유클리디안, 코사인, 피어슨, 자카드 유사도 등이 있음.

☑ 유클리디안 유사도 vs 코사인 유사도

- 문서간의 유사도를 계산 (유클리디안 유사도)



☑ 피어슨 유사도

- 문서간의 유사도를 계산

	과일이	길고	노란	먹고	바나나	사과	살은	저는	좋아요
문서1	0	0	0	1	0	1	1	0	0
문서2	0	0	0	1	1	0	1	0	0
문서3	0	1	1	0	2	0	0	0	0
문서4	1	0	0	0	0	0	0	1	1

```
1 import pandas as pd
2 pearson_sim = countvect_df.T.corr()
3 np.array(pearson_sim)
```

array([[1. , 0.5 , -0.45883147, -0.5 , 1.],
 [0.5 , 1. , 0.22941573, -0.5 , 1.],
 [-0.45883147, 0.22941573, 1. , -0.45883147, 1.],
 [-0.5 , -0.5 , -0.45883147, 1. , 1.]])

$$r_{XY} = \frac{\sum_i^n (X_i - \bar{X})(Y_i - \bar{Y})}{\sqrt{\sum_i^n (X_i - \bar{X})^2} \sqrt{\sum_i^n (Y_i - \bar{Y})^2}}$$

$$\frac{1 + 1}{\sqrt{6(\frac{1}{3})^2 + 3(\frac{2}{3})^2} \cdot \sqrt{6(\frac{1}{3})^2 + 3(\frac{2}{3})^2}} = 0.5$$

- 유클리디안은 가장 가까운 점을 찾기에 파란색의 "young"이 가장 유사한 벡터가 됨.
- 하지만, 코사인 유사도의 경우 크기가 아닌 방향성을 보기에 "mid" 혹은 "adult"가 가장 유사하게 됨.

- 문서간의 유사도를 계산

	과일이	길고	노란	먹고	바나나	사과	살은	저는	좋아요
문서1	0	0	0	1	0	1	1	0	0
문서2	0	0	0	1	1	0	1	0	0
문서3	0	1	1	0	2	0	0	0	0
문서4	1	0	0	0	0	0	0	1	1

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|} = \frac{|A \cap B|}{|A| + |B| - |A \cap B|}$$

```
1 from sklearn.metrics.pairwise import pairwise_distances
2 jac_sim = 1 - pairwise_distances(np.array(countvect_df), metric = 'jaccard')
3 jac_sim
```

array([[1. , 0.5 , 0. , 0.],
 [0.5 , 1. , 0.2 , 0.],
 [0. , 0.2 , 1. , 0.],
 [0. , 0. , 0. , 1.]])

2. 추천 알고리즘 종류

02

컨텐츠 기반 추천

- 유사도 함수 이외에도 TF-IDF와 Word2Vec을 이 용합니다.

TF-IDF

- 특정 문서 내에 특정 단어가 얼마나 자주 등장하는 지를 의미하는 단어 빈도(TF)와 전체 문서에서 특 정 단어가 얼마나 자주 등장하는지를 의미하는 역 문서 빈도(DF)를 통해서 "다른 문서에서는 등장하 지 않지만 특정 문서에서만 자주 등장하는 단어"를 찾아서 문서 내 단어의 가중치를 계산하는 방법

Word2Vec

- 워드 임베딩, 추론 기반의 방법
 - CBOW : 주변에 있는 단어들로 중간에 있는 단어들 을 예측
 - Skip-Gram: 중심 단어를 보고 어떤 주변 단어가 존 재하는지 예측하는 모델

- TF : 특정 문서 d에서의 특정 단어 t의 등장 횟수
- DF : 특정 단어 t가 등장한 문서의 수
- IDF : DF에 반비례하는 수 (idf-smoothing 여부에 따라서 결과가 달라짐)
- TF-IDF : TF와 IDF를 곱해준 값

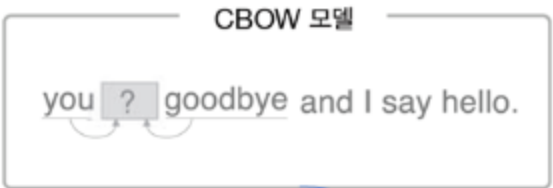
문서	내용
0	먹고 싶은 사과
1	먹고 싶은 바나나
2	길고 노란 바나나 바나나
3	저는 과일이 좋아요

TF-IDF 방법 (통계기반의 방법 - 빈도수)

TF-IDF는 특정 문서 내에 특정 단어가 얼마나 자주 등장하는 지를 의미하는 단어 빈도(TF)와 전체 문서에서 특정 단어가 얼마나 자주 등장하는지를 의미하는 역문서 빈도(DF)를 통해서 "다른 문서에서는 등장하지 않지만 특정 문서에서만 자주 등장하는 단어 " 를 찾아서 문서 내 단어의 가중치를 계산하는 방법입니다.

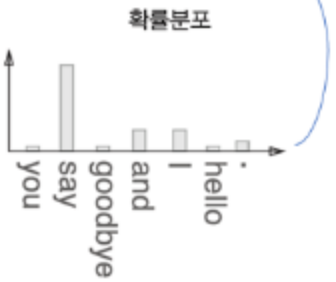
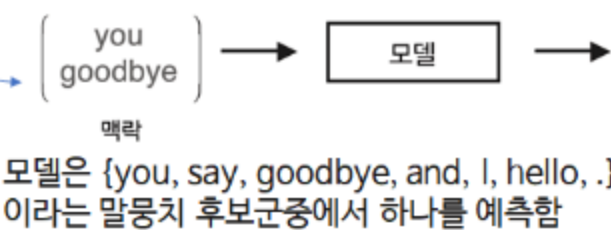
	과일이	길고	노란	먹고	바나나	사과	싶은	저는	좋아요
문서1	0	0	0	0.287682	0	0.693147	0.287682	0	0
문서2	0	0	0	0.287682	0	0	0.287682	0	0
문서3	0	0.693147	0.693147	0	0	0	0	0	0
문서4	0.693147	0	0	0	0	0	0	0.693147	0.693147

추론 : 주변 단어(맥락)이 주어졌을 때 "?"에 무슨 단어(중심단어)가 들어가는지를 추측하는 작업
예) you say goodbye and I say hello.



잘못 예측한 경우?
: 학습을 통해서 점점 모델을 개선

- 주변 단어 : 주변에 있는 단어 (you, goodbye)
- 중심 단어 : 중간에 있는 단어 (say)
- 윈도우 크기 : 주변을 몇 칸까지 볼 지에 대한 크기 (1)



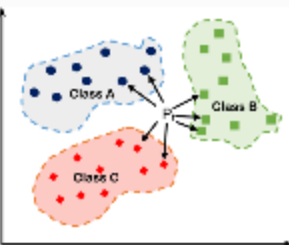
2. 추천 알고리즘 종류

03 협업 필터링 추천

- 사용자의 구매 패턴이나 평점을 가지고 다른 사람들의 구매 패턴, 평점을 통해서 추천을 하는 방법

KNN

- K Nearest Neighbors, 유사한 K 명의 Neighbors를 통해서 예측 (유사한 상품, 유사한 사용자)



1. 숫자 k와 거리 측정 기준을 선택
2. 분류하려는 샘플에서 k개의 최근접 이웃을 찾는다.
3. 가장 많이 속한 집단의 class label을 할당한다.(다수결 투표)

	Item1	Item2	Item3	Item4	Item5	Item6	mean
User1	7	6	7	4	5	4	5.5
User2	6	7	?	4	3	4	4.8
User3	?	3	3	1	1	?	2
User4	1	2	5	3	3	4	2.5
User5	1	?	1	2	3	3	2

User3과 나머지 user의 유사도 비교. ?된 곳을 제외하고 평균을 구함

사용자-아이템의 평점 행렬에서 비슷한 유저를 유사도행렬을 통해서 찾음.
(코사인, 피어슨 유사도 등)

	평균	Cosine(i, 3)	Pearson(i, 3)
사용자1	5.5	0.956	0.894
사용자2	4.8	0.981	0.939
사용자3	2	1.0	1.0
사용자4	2.5	0.789	-1.0
사용자5	2	0.645	-0.817

$$\hat{r}_{ij} = \mu_u + \frac{\sum_{v \in P_u(j)} \text{Sim}(u, v) \cdot r_{vj}}{\sum_{v \in P_u(j)} |\text{Sim}(u, v)|} = \mu_v + \frac{\sum_{u \in P_v(i)} \text{Sim}(u, v) \cdot (r_{uj} - \mu_u)}{\sum_{u \in P_v(i)} |\text{Sim}(u, v)|} \quad (2.4)$$
$$\text{Cosine}(1, 3) = \frac{6 * 3 + 7 * 3 + 4 * 1 + 5 * 1}{\sqrt{6^2 + 7^2 + 4^2 + 5^2} \cdot \sqrt{3^2 + 3^2 + 1^2 + 1^2}} = 0.956$$
$$\text{Pearson}(1, 3) = \frac{(6 - 5.5) * (3 - 2) + (7 - 5.5) * (3 - 2) + (4 - 5.5) * (1 - 2) + (5 - 5.5) * (1 - 2)}{\sqrt{1.5^2 + 1.5^2 + (-1.5)^2 + (-0.5)^2} \cdot \sqrt{1^2 + 1^2 + (-1)^2 + (-1)^2}} = 0.894$$

	아이템1	아이템6	평균	Pearson(i, 3)
사용자1	7	4	5.5	0.894
사용자2	6	4	4.8	0.939
사용자3	3.35	0.86	2	1.0
사용자4	1	4	2.5	-1.0
사용자5	1	3	2	-0.817

$$\hat{r}_{31} = \frac{7 * 0.894 + 6 * 0.939}{0.894 + 0.939} \approx 6.49$$
$$\hat{r}_{36} = \frac{4 * 0.894 + 4 * 0.939}{0.894 + 0.939} = 4$$

사용자1의 아이템1의 평점 - 사용자1의 평균 평점

$$\hat{r}_{31} = 2 + \frac{1.5 * 0.894 + 1.2 * 0.939}{0.894 + 0.939} \approx 3.35$$
$$\hat{r}_{36} = 2 + \frac{-1.5 * 0.894 - 0.8 * 0.939}{0.894 + 0.939} \approx 0.86$$

사용자3의 평균 평점

user 기반 협업 필터링일 경우 user3에 ?값을 채우기 위해 user3과 다른 user와의 유사도를 측정한다.

1. user에 따른 data의 mean값을 구한다.
2. 각 user에서 ?가 있는 값은 제외하고 코사인 유사도나 피어슨 유사도를 구한다.
3. 유사도가 1에 근사한 (user의 평점-user의 평균평점) * 유사도값의 가중평균을 통해 ?값을 채운다.

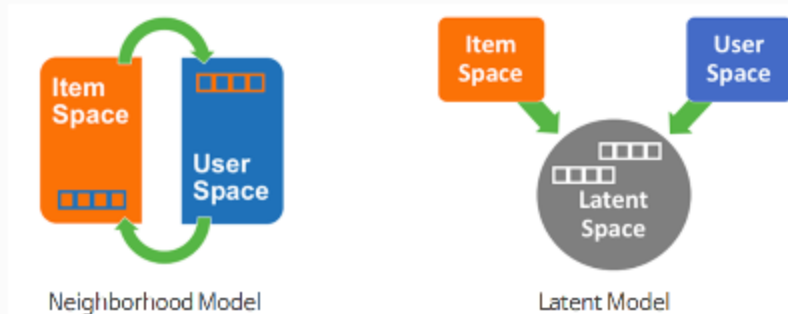
2. 추천 알고리즘 종류

03 협업필터링 추천

- 사용자의 구매 패턴이나 평점을 가지고 다른 사람들의 구매 패턴, 평점을 통해서 추천을 하는 방법

SVD, SGD, ALS

- Latent Factor Collaborative Filtering (잠재요인 협업필터링)



Rating Matrix에서 빈 공간을 채우기 위해서 사용자와 상품을 잘 표현하는 차원(Latent Factor)을 찾는 방법, 잘 알려진 행렬 분해는 추천 시스템에서 사용되는 협업 필터링 알고리즘의 한 종류.

Matrix Factorization

		Item			
		W	X	Y	Z
User	A		4.5	2.0	
	B	4.0		3.5	
	C		5.0		2.0
	D		3.5	4.0	1.0

Rating Matrix

$$=$$

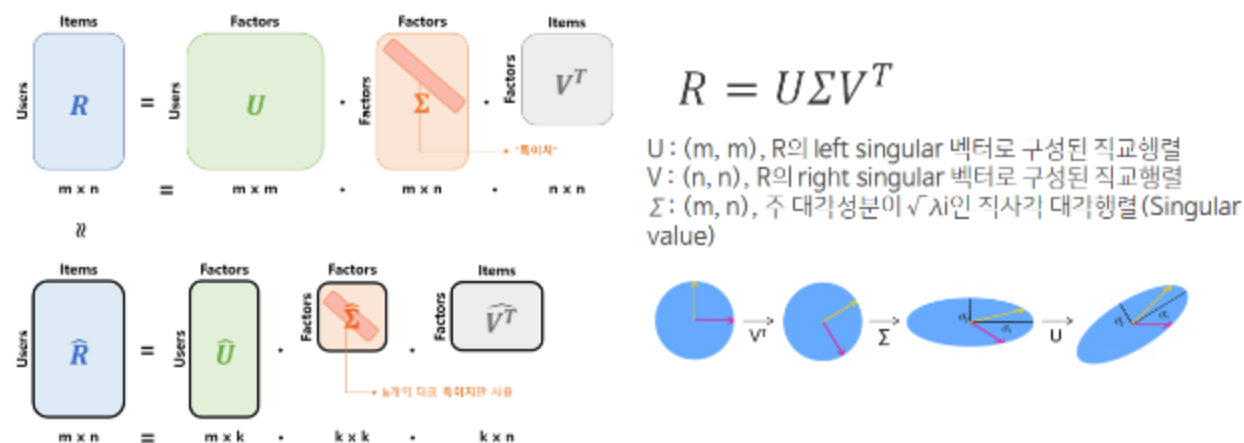
		Item			
		W	X	Y	Z
User	A	1.2	0.8		
	B	1.4	0.9		
	C	1.5	1.0		
	D	1.2	0.8		

User Matrix

$$\times$$

		Item			
		W	X	Y	Z
Item	W	1.5	1.2	1.0	0.8
	X	1.7	0.6	1.1	0.4
	Y				
	Z				

Item Matrix



- SVD 방식은 고윳값 분해(eigen value Decomposition)와 같은 행렬을 대각화 하는 방법으로, 데이터에 결측치가 없어야 한다는 한계점이 있다.

SGD Algorithm for MF

Input: training matrix V , the number of features K , regularization parameter λ , learning rate ϵ
Output: row related model matrix W and column related model matrix H

- 1: Initialize W, H to $UniformReal(0, \frac{1}{\sqrt{K}})$
- 2: **repeat**
- 3: **for random** $V_{ij} \in V$ **do**
- 4: $error = W_{i*}H_{*j} - V_{ij}$
- 5: $W_{i*} = W_{i*} - \epsilon(error \cdot H_{*j}^T + \lambda W_{i*})$
- 6: $H_{*j} = H_{*j} - \epsilon(error \cdot W_{i*}^T + \lambda H_{*j})$
- 7: **end for**
- 8: **until** convergence

- SGD((Singular Value Decomposition)를 이용한 협업 필터링은 결측치와 상관없이 진행하는 장점이 있음.
- 임의로 User Latent와 Item Latent의 초기값을 둔 뒤 기울기 하강법을 통해 모든 평점에 대해 반복하여 두 행렬의 최적값을 동시에 찾아가는 방식.

Algorithm 1 ALS for Matrix Completion

Initialize X, Y
repeat
 for $u = 1 \dots n$ **do**

$$x_u = (\sum_{i: (i,u) \in V} y_i y_i^T + \lambda I_k)^{-1} \sum_{i: (i,u) \in V} r_{ui} y_i$$
 (2)
 end for
 for $v = 1 \dots m$ **do**

$$y_v = (\sum_{i: (i,v) \in V} x_i x_i^T + \lambda I_k)^{-1} \sum_{i: (i,v) \in V} r_{iv} x_i$$
 (3)
 end for
until convergence

- ALS(Alternating Least Squares) : User Latent, Item Latent 중 하나를 고정시킨 후 다른 행렬을 최적화하고, 최적화된 행렬을 고정시킨 후 다른 행렬을 최적화하는 방식으로 convex형태로 바뀌 수렴된 행렬을 찾는 방식.
- SGD보다 수렴속도가 빠르고 병렬화가 가능한 장점이 있음.

2. 추천 알고리즘 종류

Other Recommendation Algorithms

05 Context-based 추천

- 고객이 처한 상황과 맥락에 기초한 상품 추천 방법
 - Location-based : 유저가 어떤 위치에 있는지에 따라 추천
 - Real-time or Time-Sensitive : 시간의 변화에 따라 추천

06 Community-based 추천

- 고객이 속한 커뮤니티 또는 고객의 친구 등과 같은 관계가 형성된 대상의 선호도 바탕으로 추천
 - SNS 등의 뉴스피드, SNS 네트워크 데이터 활용

07 Knowledge-based 추천

- 특정 산업 지식에 기초해 상품과 고객 특성을 파악하고 그 특성 정보를 통해 추천 상품을 선별하는 추천 방법
 - Case-based : 사용자의 니즈(현재 문제 등)와 해결책 중 가장 적합한 것 골라서 추천
 - Constraint-based : 경험이나 전문지식을 통해 만들어진 규칙에 기초한 제약 기반 추천

추천시스템

유저 관련 정보

(성별, 연령, 지역, 관심사)
유저ID, 디바이스 ID, 브라우저 쿠키 등을 사용
+ 유저행동
(페이지 방문 기록, 아이템 평가, 구매 등의 피드백 기록)

아이템 관련 정보

아이템 프로파일링
(Item's Meta Data)
: EX) 영화: 장르, 출연 배우 및 감독,
개봉년도 등
* 도메인마다 다름

유저-아이템 상호작용 정보

유저 아이템과 상호작용 할 때 로그
Explicit Feedback: 유저가 아이템에
직접적인 피드백을 '준' 경우와
Implicit Feedback: 유저가 아이템에
간접적인 피드백을 '남긴' 경우 이 있음

랭킹(Ranking)

- 유저에게 적합한 아이템 Top K개를 추천
- 평가 지표: Precision@K, Recall@K, MAP@K, nDCG@K 등

예측(Prediction)

- 유저가 아이템을 가질 선호도를 정확하게 예측
(평점 or 클릭/구매확률 등)
- 평가 지표: MAE, RMSE, AUC 등

3. 평가방법

추천시스템 평가방법



비즈니스/서비스 관점

- 추천 시스템 적용으로 인해 매출, PV의 증가
- 추천 아이템으로 인해 유저의 CTR의 상승



품질 관점

- 연관성: 추천된 아이템이 유저에게 관련이 있는가?
- 다양성: 추천된 Top-K 아이템에 얼마나 다양한 아이템이 추천되는가?
- 새로움: 얼마나 새로운 아이템이 추천되고 있는가?
- 참신함: 유저가 기대하지 못한 뜻밖의 아이템이 추천되는가?

3. 평가방법

추천시스템 평가지표 - 예측방법

01

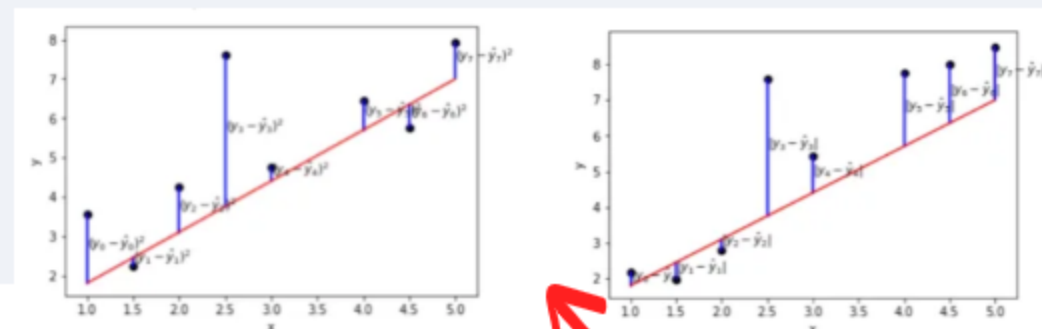
MAE & RMSE

사용자별 아이템에 대한 정답 평점과 예측 평점이 있을 때 두 평점 간 차이를 바탕으로 성능을 평가하는 방법

- MAE는 정답 평점과 예측 평점 간 절대 오차에 대해서 평균

- RMSE : 오차 제곱의 평균을 내고 루트를 취한 것, 제곱으로 인해 단위가 커지는데, 루트를 씌움으로써 실제 예측값과 유사한 단위를 가지도록 만듦.

머신러닝 회귀 성능지표와 동일합니다.



$$MAE = \frac{1}{|\hat{R}|} \sum_{\hat{r}_{ui} \in \hat{R}} |r_{ui} - \hat{r}_{ui}|$$

* $|\hat{R}|$ = 예측한 평점의 개수

$$RMSE = \sqrt{\frac{1}{|\hat{R}|} \sum_{\hat{r}_{ui} \in \hat{R}} (r_{ui} - \hat{r}_{ui})^2}$$

* $|\hat{R}|$ = 예측한 평점의 개수

Epoch	Prediction	Target
1	[0, 4, 9]	[3, 5, 7]
2	[2, 4, 2]	[3, 5, 7]
3	[3, 5, 6]	[3, 5, 7]

$$MSE = \frac{1}{N} \sum_i (pred_i - target_i)^2$$

$$RMSE = \sqrt{\frac{1}{N} \sum_i (pred_i - target_i)^2}$$

$$MAE = \frac{1}{N} \sum_i |(pred_i - target_i)|$$

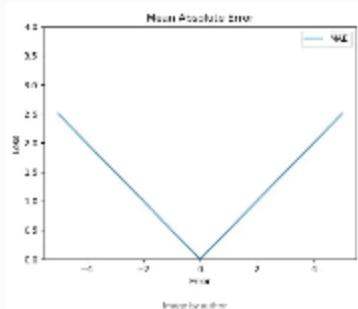
3. 평가방법

추천시스템 평가지표 - 예측방법

01

MAE & RMSE

MAE 특징

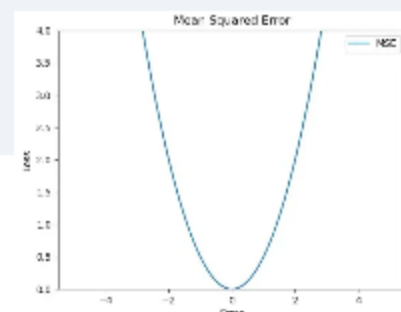


- 이상치에 둔감 혹은 강건하다.
- MAE는 위 MSE, RMSE에 비해, 오차값이 outlier의 영향을 상대적으로 크게 받지 않는다.
- 함수값에 미분 불가능한 지점이 있다.
- 모든 오차에 동일한 가중치를 부여한다. (MSE, RMSE와 대조된다)

차이점

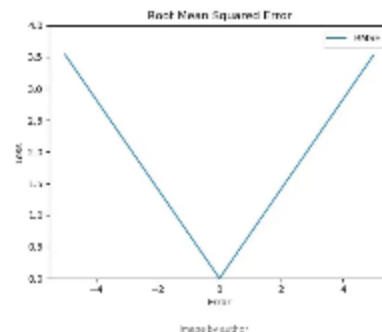
MAE는 오차들의 절댓값의 평균을 계산한다는 점에서, 모든 examples에 대한 오차에 동일한 가중치를 부여한다. 반면에, RMSE는 각 example에 제곱을 취한 뒤 평균을 구하고, 그것에 루트를 씌우는 것이기 때문에, 각 오차가 다른 가중치를 갖는다

MSE 특징



- 예측값과 정답의 차이를 제곱하기 때문에, 이상치에 대해 민감하다.
- 오차값에 제곱을 취하기 때문에 (1) 오차가 0과 1 사이인 경우에 본래보다 더 작게 반영되고, (2) 오차가 1보다 클 때는 본래보다 더 크게 반영된다.

RMSE 특징



- MSE에서 루트를 취하기 때문에, MSE의 단점이 어느 정도 해소된다.
- MSE는 부드러운 곡선형으로 오차 함수가 그려지지만, RMSE는 그 MSE에서 루트를 취하기 때문에 미분 불가능한 지점을 갖는다.
- MSE 보다 이상치에 덜 민감하다. 이 RMSE은 이상치에 대한 민감도가 MSE와 MAE 사이에 있기 때문에, 이상치를 적절히 잘 다룬다고 간주되는 경향이 있다.

3. 평가방법

추천시스템 평가지표 - 랭킹방법

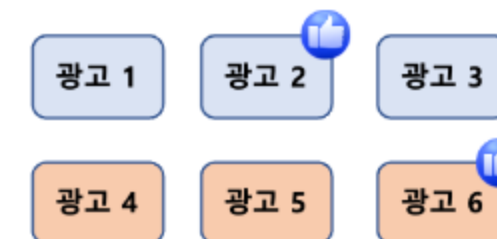
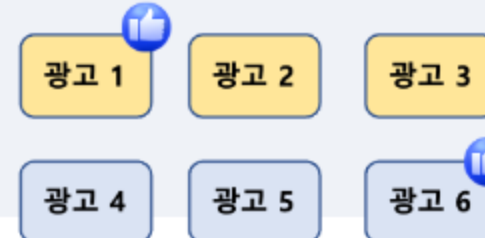
01

Precision/Recall @K

precision, recall을 k개로 확장한 것

- Precision@K : 우리가 추천한 k개 아이템 가운데 실제 유저가 관심있는 아이템의 비율

- Recall@K : 유저가 관심있는 전체 아이템 가운데 우리가 추천한 아이템의 비율



Precision@1= 1/1

유저 1을 보면, 추천시스템이 상위 광고 1개 (광고 1)을 유저 1에게 추천했고 유저 1도 실제로 선호

Precision@2= 1/2

추천시스템이 추천한 상위 광고 2개 (광고 1,2) 중 유저 1은 1개 (광고 1)만 선호

3. 평가방법

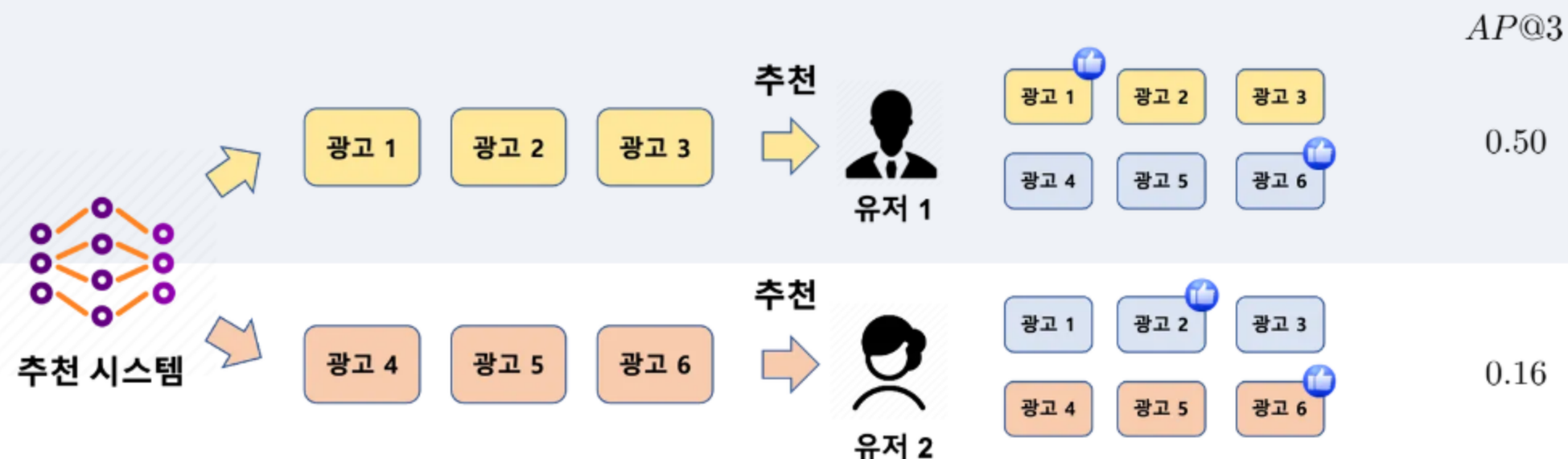
추천시스템 평가지표 - 랭킹방법

02

Mean Average Precision(MAP) @K

- AP@K: (유저 하나에 대한) Precision@1 부터 Precision@K 까지의 평균 값
- Precision@K와 달리, 관련 아이템을 더 높은 순위에 추천할수록 점수가 상승함(순서가 반영됨)

MAP@K: 모든 유저에 대한 Average Precision 값의 평균



$$MAP@K = \frac{1}{|U|} \sum_{u=1}^{|U|} (AP@K)_u = \frac{1}{2} (0.50 + 0.16) = 0.33$$

$$AP@K = \frac{1}{m} \sum_{i=1}^K Precision@i \cdot rel(i) = \frac{1}{2} \left(\frac{1}{1} \times 1 + \frac{1}{2} \times 0 + \frac{1}{3} \times 0 = 0.5 \right)$$

유저가 좋아한 아이템 수

해당 아이템을 유저가 좋아했는지 여부 (0/1)

3. 평가방법

추천시스템 평가지표 - 랭킹방법

03

NDCG@K (Normalized Dicounted Cumulative Gain@K)

추천 순위에 가중치를 두어, 상위의 랭킹 리스트가 하위 랭킹 리스트보다 확연하게 중요한 도메인에서는 유용한 평가 기준

- 다양한 관련도(1~5)를 고려하며 더 관련있는 콘텐츠는 무엇인가?에 대해 집중하여 해당 콘텐츠를 상위에 노출시키는지에 대해 평가

$$NDCG = \frac{DCG}{IDCG}$$

추천시스템의 추천 성능

이상적인 추천시스템의 추천 성능

이상적인 시스템 대비 추천 성능



Relevance

광고 1

3

광고 2

2

광고 3

0

광고 4

1

광고 5

4

가장 선호



추천 시스템

$i = 1$

광고 1

3

$i = 2$

광고 2

2

$i = 3$

광고 3

0

Relevance

$$DCG_K = \sum_{i=1}^K \frac{relevance_i}{\log_2(i+1)} = \frac{3}{\log_2(1+1)} + \frac{2}{\log_2(2+1)} + \frac{0}{\log_2(3+1)} = 4.26$$

DCG@K (Dicounted Cumulative Gain)

mAP와 마찬가지로 추천된 아이템의 순위와 relevance 점수를 고려하지만 relevance 점수가 binary(0,1)가 아니라 다양한 범위의 값을 가진다.

$$IDCG_K = \sum_{i=1}^K \frac{relevance_i^{opt}}{\log_2(i+1)} = \frac{4}{\log_2(1+1)} + \frac{3}{\log_2(2+1)} + \frac{2}{\log_2(3+1)} = 6.89$$

IDCG@K (Ideal Dicounted Cumulative Gain)

relevance 점수가 높은 순서대로 추천하는 것이 이상적으로 이상적인 수치 계산

3. 평가방법

추천시스템 평가지표 - 랭킹방법

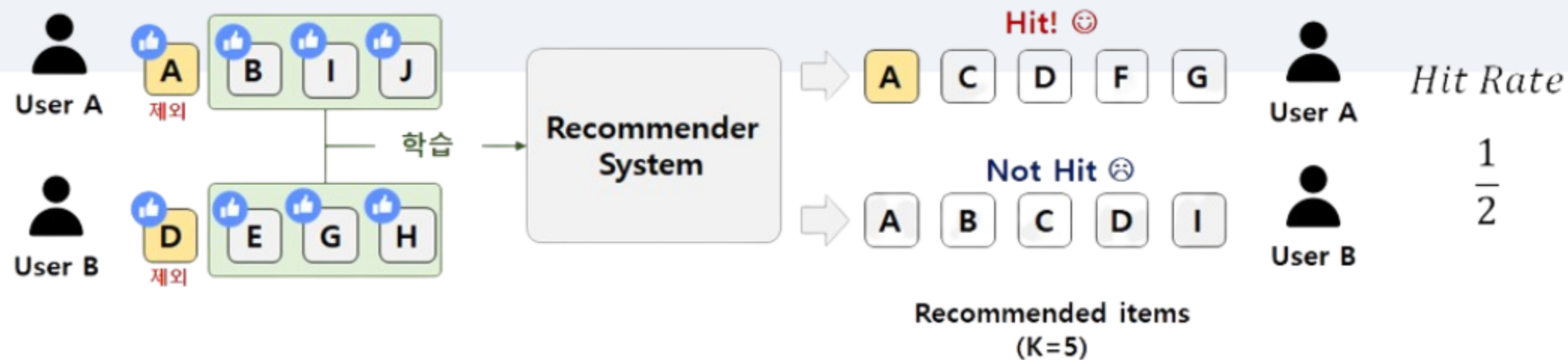
04

Hit Rate@K

전체 사용자 수 대비 적중한 사용자 수를 의미
(적중률)

*구하는 방법

- 1) 사용자가 선호한 아이템 중 1개를 제외한다.
- 2) 나머지 아이템들로 추천 시스템을 학습한다.
- 3) 사용자별로 K개의 아이템을 추천하고, 앞서 제외한 아이템이 포함되면 Hit이다.
- 4) 전체 사용자 수 대비 Hit한 사용자 수 비율을 구하면 Hit Rate가 된다.



$$\text{Hit Rate} = \frac{\# \text{ of Hit User}}{\# \text{ of User}}$$

3. 평가방법

추천시스템 평가지표 - 랭킹방법

etc

Coverage

: 전체 사용자 및 아이템 중에서 추천시스템이 놓치고 있는 부분이 없는지를 판단

전체 사용자 및 아이템 중에서 한 번이라도 추천 결과가 생성된 사용자 또는 추천 결과에 포함된 아이템의 비율을 의미

ex) 총 50 개의 아이템 카탈로그 중 추천 결과에 포함된 아이템의 개수가 30 이라면,
 $\text{item-space coverage} = 30/50 = 0.6$ / accuracy와 coverage는 Trade-Off 관계임.

Confidence & Trust

- Confidence는 추천 결과를 제공하는 시스템의 신뢰성 (예측된 평균값이 같더라도 표준편차가 더 적으면, 더 높은 confidence 값을 가짐)

- Trust는 사용자가 추천 결과에 대해 가지고 있는 믿음 (사용자가 좋아했던 아이템을 다시 추천하는 것은 새로운 측면에서는 유용하지 않을지는 몰라도 trust를 증가시키는 효과가 있다함)

Serendipity

: 성공적인 추천 결과로부터 사용자가 느끼는 놀라움의 정도

$\text{Serendipity} = \text{novelty} + \text{relevance} + \text{unexpectedness}$

Novelty

: 사용자가 알지 못하거나 이전에 본 적이 없는 새로운 추천을 사용자에게 제공할 가능성을 의미
Novelty가 높은 추천은 종종 사용자가 이전에는 알지 못했던 취향에 대한 새로운 발견을 제공함.

Diversity

: 추천 결과가 얼마나 다양한 아이템들로 이루어졌는지를 의미

Robustness & Stability

: 추천시스템에 가해질 수 있는 적대적인 공격(fake ratings, ...)에 대한 견고함과 안정성을 평가

Scalability

: 추천시스템이 대용량 데이터 및 트래픽을 효과적이고 효율적으로 처리할 수 있는지를 평가

4. 설계 및 구현

데이터 탐색

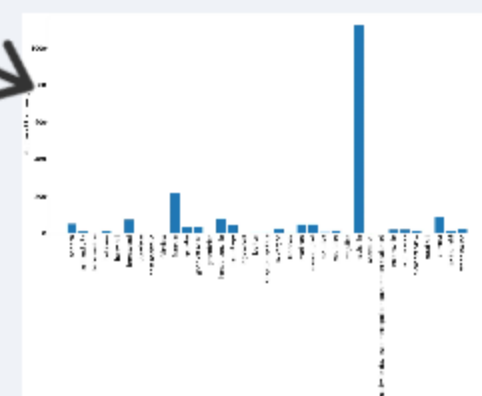
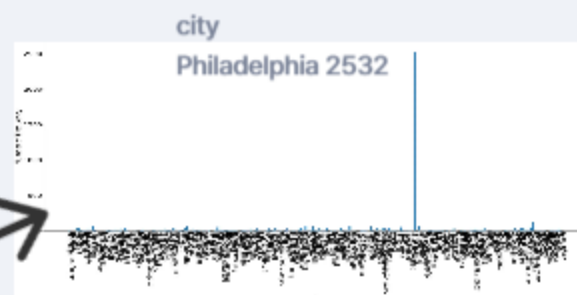
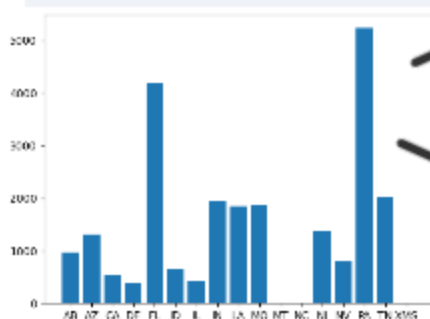
Yelp academic dataset 을 다운받고
데이터를 살펴본 후, 추천시스템에 맞는 형태로 전처리 합니다.



Business

레스토랑의 rating 점수, 좌표, 도시, 리뷰 갯수
등의 데이터

```
1 df.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 23571 entries, 0 to 23570
Data columns (total 16 columns):
#   Column      Non-Null Count  Dtype
---  -
0   business_id 23571 non-null  object
1   name        23571 non-null  object
2   address     23167 non-null  object
3   city        23571 non-null  object
4   state       23571 non-null  object
5   postal_code 23551 non-null  object
6   latitude    23571 non-null  float64
7   longitude   23571 non-null  float64
8   stars       23571 non-null  float64
9   review_count 23571 non-null  int64
10  is_open     23571 non-null  int64
11  attributes  23343 non-null  object
12  categories  23571 non-null  object
13  hours       21235 non-null  object
14  isRestaurant 23571 non-null  int64
15  rank        23571 non-null  float64
dtypes: float64(4), int64(3), object(9)
memory usage: 2.9+ MB
```



Review

리뷰글, rating, cool, funny, useful 등등의
점수 데이터

```
1 review_df.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2154455 entries, 0 to 2154454
Data columns (total 10 columns):
#   Column      Dtype
---  -
0   review_id   object
1   user_id     object
2   business_id object
3   stars       float64
4   useful      int64
5   funny       int64
6   cool        int64
7   text        object
8   date        object
9   isRestaurant int64
dtypes: float64(1), int64(4), object
memory usage: 164.4+ MB
```

User

사용자ID, name, 남긴 리뷰 수, 친구, 평균평점
등 사용자 데이터

```
3 user_df.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 892975 entries, 0 to 892974
Data columns (total 23 columns):
#   Column      Non-Null Count  Dtype
---  -
0   user_id     892975 non-null  object
1   name        892974 non-null  object
2   review_count 892975 non-null  int64
3   yelping_since 892975 non-null  object
4   useful      892975 non-null  int64
5   funny       892975 non-null  int64
6   cool        892975 non-null  int64
7   elite       69063 non-null   object
8   friends     892975 non-null  object
9   fans        892975 non-null  int64
10  average_stars 892975 non-null  float64
11  compliment_hot 892975 non-null  int64
12  compliment_more 892975 non-null  int64
13  compliment_profile 892975 non-null  int64
14  compliment_cute 892975 non-null  int64
15  compliment_list 892975 non-null  int64
16  compliment_note 892975 non-null  int64
17  compliment_plain 892975 non-null  int64
18  compliment_cool 892975 non-null  int64
19  compliment_funny 892975 non-null  int64
20  compliment_writer 892975 non-null  int64
21  compliment_photos 892975 non-null  int64
22  isRestaurant 892975 non-null  int64
dtypes: float64(1), int64(17), object(5)
memory usage: 156.7+ MB
```

4. 설계 및 구현

데이터 형식 맞추기

user_id	business_id	user_rating
mh_-eMZ6K5RLWhZyISBhwA	XQfwVwDr-v0ZS3_CbbE5Xw	3.0
laee7y6zdSB3B-kRC04z1w	XQfwVwDr-v0ZS3_CbbE5Xw	2.0
ejFxLGqQcWNLdNByJllhnQ	XQfwVwDr-v0ZS3_CbbE5Xw	4.0
f7xa0p_1V9lx53ilGN5Sug	XQfwVwDr-v0ZS3_CbbE5Xw	3.0
dCooFVck8M1nVaQqcTL3Q	XQfwVwDr-v0ZS3_CbbE5Xw	2.0

확습 데이터셋 처리

User CF - row : 사용자, column : 아이템, values : 평가점수
Item CF - row : 아이템, column : 사용자, values : 평가점수

```
piv =  
merged.pivot_table(index='user_id',  
columns='business_id',  
values='user_rating')
```

- 사용자 평점 여러개 준 음식점, 평균으로 처리

```
1 ratings_mean_count, ratings_mean_count_b
```

```
(  
  user_id      business_id  user_m  user_c  
0  ---2PmXbF47D870stH1jqA  0ZsqzHu1HHkDdIKoivi5g  5.0  1  
1  ---2PmXbF47D870stH1jqA  1An4DxtMmvySe0HX4viRCA  5.0  3  
2  ---2PmXbF47D870stH1jqA  KP50ncF2jhT7_J1phHPPww  5.0  1  
3  ---2PmXbF47D870stH1jqA  L5LLNORafiV1Z9cddzvuCw  5.0  1  
4  ---2PmXbF47D870stH1jqA  MxRZHZoDVVnN7EvMAHf1EA  5.0  1  
...  
2081541  zzy6clesq9K8AGo74trKAA  DNmdZVVfhus2H-dH0m13HA  4.0  1  
2081542  zzyTsajskrf_Ha8gU4HmTA  QAMJ1JQ7SM0-C5xUN37ilw  5.0  1  
2081543  zzz-M4QvkEpUWWPL9RTzLA  8BE1pKZu-ddcptxDbaMwLQ  4.0  1  
2081544  zzz-M4QvkEpUWWPL9RTzLA  _aZAD5CDmaYAB09KgddpPQ  1.0  1  
2081545  zzz-M4QvkEpUWWPL9RTzLA  hn3Rg2JrhQoDJBhrpuwWg  2.0  1  
...  
[2081546 rows x 4 columns],  
  business_id  r_mean  r_count  
0  ---kPU91CF4Lq2-WIRu9Lw  4.5  1  
1  --0iUa4sNDFiZFrAdIWhZQ  3.0  1  
2  --81b0sAAxjKRoYsBFL-PA  3.0  1  
3  --ZvRH2X2QXBFdCilbirsw  4.5  1  
4  --epgcb7xHGwJ-4PueSLAw  3.0  1  
...  
23566  zzFCdBSW27eKFg-xG7cqAg  4.0  1  
23567  zzXRdzrVhfNWPd2MeyWeA  2.0  1  
23568  zzbZtgPYZS8sTlWQH6DwEw  3.0  1  
23569  zziDpuuJw-Km1J4BaGpBKA  3.5  1  
23570  zznJox6-nmXlGYNWgTDwQQ  1.5  1  
...  
[23571 rows x 3 columns])
```

merge

```
1 # Join the two dataframes on the scis_c_nm columns  
2 merged = user_rv.merge(item, left_on = 'business_id', right_on = 'business_id', suffixes=('_user', '_item'))  
3 merged.rename(columns = {'stars':'user_rating'}, inplace = True)
```

```
1 merged.head()
```

	user_id	business_id	user_rating	r_mean
0	mh_-eMZ6K5RLWhZyISBhwA	XQfwVwDr-v0ZS3_CbbE5Xw	3.0	3.0
1	laee7y6zdSB3B-kRC04z1w	XQfwVwDr-v0ZS3_CbbE5Xw	2.0	3.0
2	ejFxLGqQcWNLdNByJllhnQ	XQfwVwDr-v0ZS3_CbbE5Xw	4.0	3.0
3	f7xa0p_1V9lx53ilGN5Sug	XQfwVwDr-v0ZS3_CbbE5Xw	3.0	3.0
4	dCooFVck8M1nVaQqcTL3Q	XQfwVwDr-v0ZS3_CbbE5Xw	2.0	3.0

↓ pivot

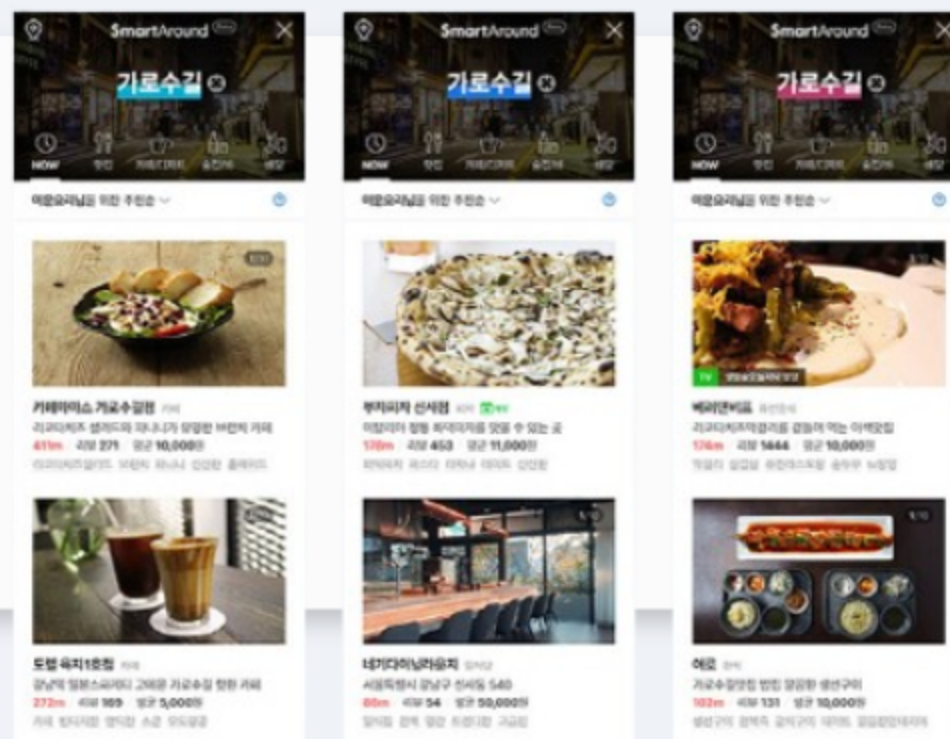
User CF - row : 사용자, column : 아이템, values : 평가점수 (120184, 2534)
Item CF - row : 아이템, column : 사용자, values : 평가점수
음식점 도시 Philadelphia로 한정 (사용자수 : 279466, 음식점수 : 2534)

4. 설계 및 구현

베스트 셀러, 위치 기반 추천

01
베스트 셀러 추천

- 카테고리 내 인기 레스토랑 순으로 추천



05

Context-based 추천의 위치기반 추천

- 레스토랑의 위도와 경도의 데이터가 있음
- 사용자 위치에 대해 레스토랑 추천

4. 설계 및 구현

협업필터링

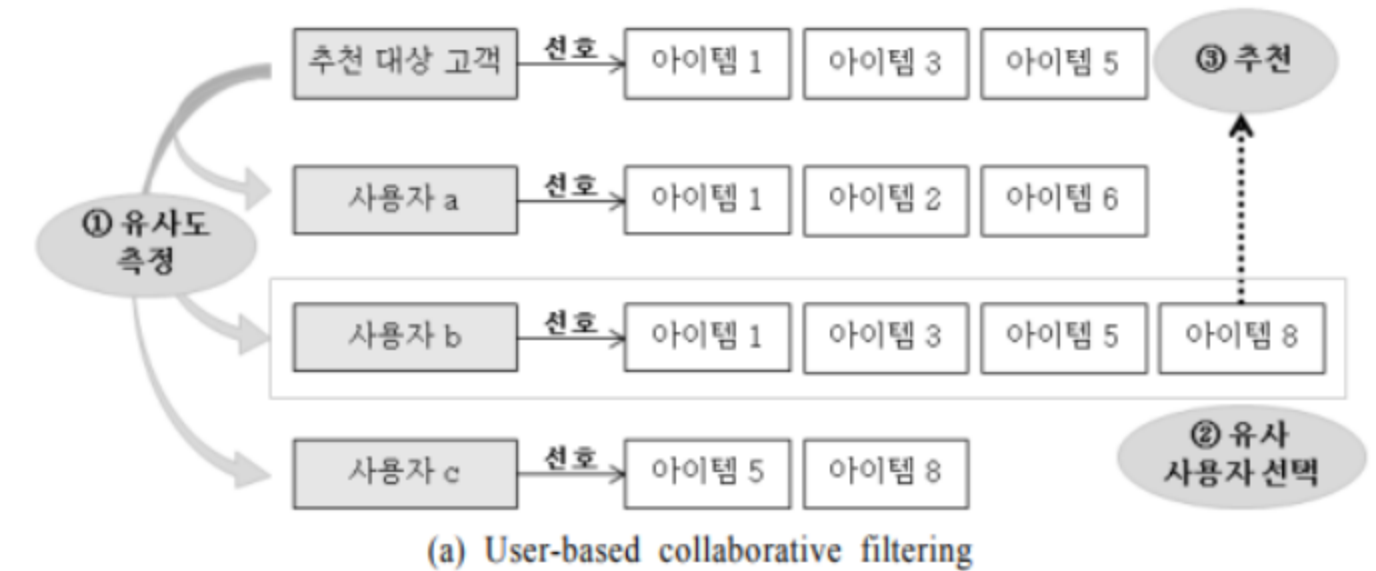
03 협업필터링

- 사용자 기반 협업 필터링: 타겟 사용자와 유사한 사용자를 찾아 유사 사용자가 좋아하는 항목을 추천.
- 아이템 기반 협업 필터링: 대상 사용자가 좋아요를 누른 아이템과 유사한 아이템을 찾아 추천.

- 사용자기반



- 아이템 기반



(a) User-based collaborative filtering



(b) Item-based collaborative filtering

Figure 3. The algorithm of collaborative filtering

4. 설계 및 구현

추천시스템 적용

01

베스트셀러

top-N 정확도 계산

사용자별로 10개의 추천 아이템을 생성

인자인 predictions로는 SVD의 test의 리턴값을 사용

```
def get_top_n(predictions, n=10):
```

```
    top_n = defaultdict(list)
```

```
    # predictions의 각 레코드는
```

```
    # (사용자 ID, 음식점 ID, 실제 평점, 예측 평점)으로 구성
```

```
    for uid, iid, true_r, est, _ in predictions:
```

```
        # 사용자별로 음식점ID와 예측 평점을 같이 저장
```

```
        top_n[uid].append((iid, est))
```

```
    # 위에서 채워진 top_n을 사용자ID를 가지고 예측평점이 높은 n개의 영화로 교체
```

```
    for uid, user_ratings in top_n.items():
```

```
        user_ratings.sort(key=lambda x: x[1], reverse=True)
```

```
        top_n[uid] = user_ratings[:n]
```

```
    return top_n
```

02

협업필터링

유사도 계산, scipy의 csr_matrix

```
piv_sparse
```

```
= sp.sparse.csr_matrix(piv_norm.values)
```

```
item_similarity
```

```
= cosine_similarity(piv_sparse)
```

```
user_similarity
```

```
= cosine_similarity(piv_sparse.T) # 전치
```

03

위치기반 X

메모리 부족으로 실행불가

```
def top_item(item_nm):
```

```
    count = 1
```

```
    print('Similar shows to {} include:\n'.format(item_nm))
```

```
    result = item_sim_df.loc[~item_sim_df.index.isin([item_nm]),  
item_nm].sort_values(ascending = False)[:10]
```

```
    for item, score in result.items():
```

```
        print('No. {}: {}({:.2f})'.format(count, item , score))
```

```
        count +=1
```

```
def top_users(user):
```

```
    if user not in piv_norm.columns:
```

```
        return('No data available on user {}'.format(user))
```

```
    print('Most Similar Users:\n')
```

```
    result = user_sim_df.sort_values(by=user, ascending=False).loc[:,user  
[1:11]
```

```
    for user, sim in result.items():
```

```
        print('User #{0}, Similarity value: {1:.2f}'.format(user, sim))
```

*페이지 내 인물사진은 샘플이미지 입니다.

4. 설계 및 구현

추천시스템 적용

음식점 유사도는 높지 않아서 카테고리 유사한 음식점 추천(Scikit-Learn의 NearestNeighbors를 이용)

```
1 r_data = df[['business_id', 'name', 'categories']] # id, title, 장르 와 맞춰줌, 필라델피아
2
3 # 카테고리 원-핫 벡터(One-Hot Vector)로 변경
4 from sklearn.feature_extraction.text import CountVectorizer
5
6 cv = CountVectorizer()
7 cat = cv.fit_transform(r_data.categories)
8 cat = pd.DataFrame(cat.toarray(),
9 | | | | | columns = list(sorted(cv.vocabulary_.keys(), key = lambda x : cv.vocabulary_[x]))
0 )
1 cat.head(2)
```

	acai	active	adoption	adult	advising	afghan	african	airport	american	arabic	...	waffles	weddi
0	0	0	0	0	0	0	0	0	0	0	...	0	
1	0	0	0	0	0	0	0	0	1	0	...	0	

```
1 # Scikit-Learn의 NearestNeighbors를 이용
2
3 from sklearn.neighbors import NearestNeighbors
4
5 nbrs = NearestNeighbors(n_neighbors=100).fit(cat1)
6 nbrs
```

```
1 # SvdHTo4A_5eA8QsfelTPdg 인덱스
2 r_data = r_data.reset_index()
3 r_data[r_data.business_id == 'SvdHTo4A_5eA8QsfelTPdg']
```

	index	business_id	name	categories
1070	9919	SvdHTo4A_5eA8QsfelTPdg	Food for All Market	health markets, specialty food, restaurants, b...

```
14 dis, ind = nbrs.kneighbors([a])
15 recommendations = r_data.loc[ind[0], ['name']]
16 recommendations['distance'] = dis[0]
17 recommendations
```

/usr/local/lib/python3.10/dist-packages/sklearn/base.py:439: UserWarning: warn()

	name	distance
1070	Food for All Market	0.000000
99	The Pink Dolphin	2.236068
1121	Mariposa Food Co-op	2.449490
188	Essene Market & Café	2.449490
1181	Dessertcrazy	2.449490
...	...	...
1701	Auntie Anne's	3.000000
273	Tartes Pastry Shop	3.000000
674	Pagano's Steaks	3.000000
2272	Subway	3.000000
1410	New York Gyro Halal Food Cart	3.000000

```
1 recommendations.groupby('name').count().sort,
```

	cnt
Subway	7
PrimoHoagies	2
Amoroso's	1
Pagoda Noodle Cafe	1
Salam Cafe and Market	1
...	...

```
1 # 음식점 램덤
2 print(merged['business_id'].sample(1))
3 top_item(list(merged['business_id'].sample(1))[0])
```

```
140661 SvdHTo4A_5eA8QsfelTPdg
Name: business_id, dtype: object
Similar shows to bgxDswHldFPOGoQpNfyAAw include:
```

No. 1: ORSws84Im4CsZ-W9biXv2g(0.05)
No. 2: VBONwHcgkxYTHfxmdKGhkW(0.05)
No. 3: WbxPdQ_PgVvCI462IOvPEQ(0.04)
No. 4: CnL9asvqi5hwXc1sj7bqig(0.03)
No. 5: Q81Xj-eCVxi8mttVMn2UiQ(0.03)
No. 6: 1FeB0RbAz9kdoZsTf05arw(0.03)

```
11 print('입력된 음식점 : ', list(r_data[r_data.business_id == 'SvdHTo4A_5eA8QsfelTPdg'].name))
12 print(list(r_data[r_data.business_id == 'SvdHTo4A_5eA8QsfelTPdg'].name))
13 print(list(r_data[r_data.business_id == 'SvdHTo4A_5eA8QsfelTPdg'].name))
14 print(list(r_data[r_data.business_id == 'SvdHTo4A_5eA8QsfelTPdg'].name))
15 print(list(r_data[r_data.business_id == 'SvdHTo4A_5eA8QsfelTPdg'].name))
16 print(list(r_data[r_data.business_id == 'SvdHTo4A_5eA8QsfelTPdg'].name))
```

입력된 음식점 : ['Food for All Market']
['Rio Brazilian Steak Truck']
["Keating's Rope & Anchor"]
["Dunkin'"]
['Varalli']
["Jimmy John's"]

```
1 print(merged['user_id'].sample(1))
2 top_users(list(merged['user_id'].sample(1))[0])
```

```
250995 Qp4MYYSB5_R2r2JsEOHvKA
Name: user_id, dtype: object
Most Similar Users:
```

User #9fP3KiivpFVYcnqgD3aZJw, Similarity value: 0.61
User #yi0j1LB759eX9V8j5LLuPw, Similarity value: 0.57
User #lVqXMGaA-kl_-jlzA6tIrg, Similarity value: 0.54
User #WjDSSPF0u9JrROXmidZRpQ, Similarity value: 0.53
User #bq1r3VN1PpII_A7ndvY8qg, Similarity value: 0.51
User #hrUCdI00LPY3QH_4LApD6g, Similarity value: 0.51
User #ekdwfMj3KZQJ72pXx1mS-Q, Similarity value: 0.51
User #9ekoDrEC4G3Wcl5Vd0Z_Jg, Similarity value: 0.51
User #_J2VfQfTOfNuZA3Yb1_jQ, Similarity value: 0.51
User #Ss6Fz9oK7HC0vYlIxpDwEw, Similarity value: 0.49

4. 설계 및 구현

모델평가

협업필터링(SVD모델 - 서프라이즈라이브러리)

```
from surprise import SVD
from surprise import NormalPredictor
from surprise.model_selection import GridSearchCV
# 하이퍼 파라미터 그리드 검색을 해서 최적의 파라미터를 찾는다
# scikit-learn의 GridSearchCV와 아주 흡사
# - n_factors: 축소 차원수 (앞 그림에서 시그마 행렬의 차원수 - r)
# - n_epochs: 전체 데이터 셋을 훈련시키는 횟수
# - lr_all: 학습률
```

```
param_grid = {
    'n_epochs': [20, 30],
    'lr_all': [0.005, 0.010],
    'n_factors': [50, 100]
}
```

```
# 3-폴드 교차검증을 하고 두 개의 비용함수를 사용: RMSE와 MAE (Mean Absolute Error)
gs = GridSearchCV(SVD, param_grid, measures=['rmse', 'mae'], cv=3)
gs.fit(data)
```

```
# RMSE
print("Best RMSE score attained: ", gs.best_score['rmse'])
print("Best RMSE params: ", gs.best_params['rmse'])
# MAE
print("Best RAE score attained: ", gs.best_score['mae'])
print("Best RAE params: ", gs.best_params['mae'])
```

```
Best RMSE score attained: 1.1694221266149925
Best RMSE params: {'n_epochs': 20, 'lr_all': 0.005, 'n_factors': 50}
Best RAE score attained: 0.9256103952256884
Best RAE params: {'n_epochs': 20, 'lr_all': 0.005, 'n_factors': 50}
```

5. 결과

모델평가

협업필터링(SVD모델 - 서프라이즈라이브러리)

```
# (베스트 파라미터 모델 성능 평가)
svd = gs.best_estimator['rmse']
trainset = data.build_full_trainset()
svd.fit(trainset)
```

```
# Train/Test Split로 훈련하고 성능 평가
from surprise import accuracy
from surprise.model_selection import train_test_split
```

```
trainset, testset = train_test_split(data, test_size=.25)
svd = SVD()
svd.fit(trainset)
predictions = svd.test(testset)
# Then compute RMSE
accuracy.rmse(predictions)
testset[0:10]
```

RMSE :

```
RMSE: 1.1726
[('nnrflKvAUNsjg8Cv7FaxsA', 'BxfvdHqETU8jWYUjx50L2A', 3.0),
 ('3632AJBTgbXQDjQbo9UxNA', 'AbtYqBGuviWmRlgUdc3XA', 2.0),
 ('1b3uXHSWeBcd2kK-BKLGSw', '609Lr-Hvo3sr9andiim0JA', 5.0),
 ('SIDCG-Mkn3xb1cb0fpn73Q', 'Sv1MEZP-nMfp8SnEDhwYEA', 5.0),
 ('7JL00Q1qD1v2o3Kqk8918w', 'WP9GAuhvmUhm8NAxHchgrQ', 4.0),
 ('XXMiPQE1UWUErpt1J_4UmQ', 'zYnY0zh1G9jPoX9ANecEUw', 1.0),
 ('mcoDFbdugjc-hqgA-2vHZw', 'q1Sf5ojuYb09h71NumGUQA', 3.0),
 ('jzL0XBmgZuQzlaonKfk17A', 'eM1N8nn70jjKq81ziikYwDA', 4.0),
 ('mJCK1qpsCkKePozYlr1T-g', 'x39G7-aTCVh-972fgogKtA', 4.0),
 ('_qlrDLyU5C11-8zjYSokDA', 'dUrtFfHBz15Q4GiyDFAduw', 5.0)]
```

```
# top-N 정확도 계산
# 사용자별로 10개의 추천 아이템을 생성
```

surprise의 LeaveOneOut 모듈 : 사용자별로 평점 정보를 하나씩 테스트셋으로 저장하고 이를 나중에 Top N 추천 정확도 계산에 사용.

```
from surprise.model_selection import LeaveOneOut
# 각 사용자별로 하나의 평점만을 테스트셋에 보관 (n_splits가 1 즉 1개의 폴드)
LOOCV = LeaveOneOut(n_splits=1, random_state=1)
ratingCutoff = 4.0
```

```
# 훈련 데이터셋으로 학습
svd.fit(trainSet)
# 훈련 데이터셋에 있지 않은 모든 데이터를 테스트셋으로 리턴
bigTestSet = trainSet.build_anti_testset()
# 위에서 리턴된 데이터에 대해 평점 예측
allPredictions = svd.test(bigTestSet)
```

```
# 각 사용자별로 아직 평점이 없는 음식저들을 대상으로
# top 20 평점 리스트를 추출
topNPNredicted = get_top_n(allPredictions, n=20)
```

```
# 예를 들어 사용자 '1'의 추천 리스트를 확인
for tnp in topNPNredicted['1']:
    # tnp는 tuple로 음식점ID와 평점으로 구성
    print(tnp)
```

```
~
13 # top N 정확도를 계산
14 hits = 0
15 total = 0
16
17 # 남겨졌던 데이터의 사용자들을 대상으로 cutoffRating이상의 평점을 갖는 아이템이 추천되었는지 확인
18 for userID, business_ID, trueRating in testSet:
19     # 높은 평점이 매칭되는 경우만 고려
20     if (trueRating >= ratingCutoff):
21         # 사용자의 추천 리스트를 확인
22         for b_ID, predictedRating in topNPNredicted[userID]:
23             # 해당 사용자의 추천 아이템에 해당 음식점이 존재하는 경우에만 카운트
24             if (business_ID == b_ID):
25                 hits += 1
26                 break
27         total += 1
28
29 print(hits/float(total)+100.)
```

5.결과

추천시스템 적용 예시



- 1 사용자 기반 유사한 사용자 추천(팔로잉, 네트워크 형성)
- 2 검색내용 기반 연관상품 추천
- 3 개인 선호도 기반 협업필터링 추천

(프로젝트 후기)

- ✓ 여러 알고리즘을 이용해 보았지만, 하나의 프로젝트로 구현해보지 못한 아쉬움이 남음
- ✓ 다운로드 받은 데이터가 광범위하고 다양해서 추천시스템에 적용해볼 수 있는 것이 더 있었는데, 적용해보지 못함



recommend restaurant

2023.12.09 인공지능추천시스템 발표

감사합니다.

국민대학교 소프트웨어융합대학원
K2022531 최혜정